Assignment 2

CS 6117 (Fall 2025)

Exercise 1. Define an algebraic theory Th which has types $\alpha, \beta, \tau, \rho$ such that for every model M of Th in a category $\mathbf{C}$ with finite products, $[\![\tau]\!]_M$ is a terminal object in $\mathbf{C}$ and $[\![\alpha]\!]_M \times [\![\beta]\!]_M$ is isomorphic to $[\![\rho]\!]_M$. Check that Th does indeed satisfy these properties.

---

name clash?

The signature Sg of Th consists of the types $\alpha, \beta, \tau, \rho$, a constant symbol τ, function symbols $\pi_\alpha : \rho \to \alpha$ and $\pi_\beta : \rho \to \beta$, and a function symbol $\langle -, - \rangle : \alpha, \beta \to \rho$. The axioms Ax are:

- $x : \rho \vdash \langle \pi_\alpha(x), \pi_\beta(x) \rangle = x : \rho$ (uniqueness of pairing),
- $x : \alpha, y : \beta \vdash \pi_\alpha(\langle x, y \rangle) = x : \alpha$ (first projection),
- $x : \alpha, y : \beta \vdash \pi_\beta(\langle x, y \rangle) = y : \beta$ (second projection),
- $x : \tau, y : \tau \vdash x = y : \tau$ (uniqueness for terminal type).

this is all true but could use a bit more detail to spell out

In any model M in $\mathbf{C}$ with finite products, $[\![\tau]\!]_M$ is interpreted as the terminal object because the axiom forces all elements to be equal, and the constant provides the unique morphism from any object to 1. The types α, β, ρ with the pairing and projections model the product, so $[\![\rho]\!]_M \cong [\![\alpha]\!]_M \times [\![\beta]\!]_M$ by the universal property of products.

Exercise 2. In this exercise, we consider examples of algebraic theories that are closer to programming language design.

1. Consider an algebraic theory ND (non-determinism) which has a single type C (for computation) and a signature with one binary (2-ary) operation **choose**, and a nullary (0-ary) operation $\mathbf{ret}_n$ for every $n \in \mathbb{N}$.

   Intuitively, we think of $\mathbf{choose}(p_1, p_2)$ as a program that non-deterministically chooses between running p_1, or running p_2, while $\mathbf{ret}_n$ is the operation that simply returns $n \in \mathbb{N}$ as the output. Write down axioms stating that **choose** is associative, commutative, and idempotent.

2. Consider an algebraic theory $GS2$ (global state) which has a single type C and four unary (1-ary) operations $\mathbf{put}_{a,c}$ and $\mathbf{put}_{b,c}$ for $c \in \{tt, ff\}$, two

binary operations $\mathbf{get}_a$ and $\mathbf{get}_b$, and a null-ary (0-ary) operation $\mathbf{ret}_n$ for every $n \in \mathbb{N}$.

Intuitively, we think of a and b as Boolean-valued global variables. The operation $\mathbf{put}_{a,tt}(p)$ writes the value tt (true) to the variable a, and then continues running p. The operation $\mathbf{get}_a(p_t, p_f)$ reads from the variable a, and runs p_t or p_f depending on whether the variable a holds tt or ff.

Translate the following properties into axioms for the put and get operations. For a given location l:

(a) Reading a value c, writing a value c, and then continuing is the same as continuing.

(b) Reading a value c, reading a value c', and then continuing is the same as reading a value c and continuing.

(c) Writing a value c, writing a value c', and continuing is the same as writing a value c' and then continuing.

(d) Writing a value c and then reading a value c' and continuing is the same as writing a value c and then continuing.

For two different locations $l \neq l'$:

(a) Reading a value c from l and then reading a value c' from l' is the same as reading a value c' from l' and then reading a value c from l.

(b) Writing a value c to l and then writing a value c' to l' is the same as writing c' to l' and then writing c to l.

(c) Writing a value c to l and then reading a value c' from l' is the same as reading a value from c' from l' and writing a value c to l.

These equations form the axioms for global state. In fact, it is possible to show that they are complete—no new axioms can be added without equating all programs.

Algebraic theories can be combined in several ways. The *sum* of theories $ND + GS2$ takes the union of the signatures and axioms. The *tensor* of theories $ND \otimes GS2$ is obtained by taking the sum, and then adding axioms stating that all operations of ND *commute* with operations of $GS2$; for instance, non-deterministic choice followed by reading is the same as reading followed by non-deterministic choice, and similarly for all other pairs of operations. In fact, the theory $GS2$ can be obtained as the tensor of two copies of the theory GS of global state with a single variable. The category of models also satisfies a neat relation:

$$Mod(ND \otimes GS2, \mathbf{Set}) \simeq Mod(ND, Mod(GS2, \mathbf{Set})) \simeq Mod(GS2, Mod(ND, \mathbf{Set})).$$

---

1. The theory ND has type C and operations $\mathbf{choose} : C, C \to C$, $\mathbf{ret}_n : C$ for each $n \in \mathbb{N}$. Axioms:

- Associativity: $p_1 : C, p_2 : C, p_3 : C \vdash \mathbf{choose}(\mathbf{choose}(p_1, p_2), p_3) = \mathbf{choose}(p_1, \mathbf{choose}(p_2, p_3)) : C$.
- Commutativity: $p_1 : C, p_2 : C \vdash \mathbf{choose}(p_1, p_2) = \mathbf{choose}(p_2, p_1) : C$.
- Idempotence: $p : C \vdash \mathbf{choose}(p, p) = p : C$.

These axioms reflect the semantics of non-deterministic choice in programming, where order doesn't matter, and choosing the same program twice is equivalent to choosing it once.

2. The theory $GS2$ has type C, operations $\mathbf{put}_{l,c} : C \to C$ for $l \in a, b$, $c \in tt, ff$, $\mathbf{get}_l : C, C \to C$ for $l \in a, b$, and $\mathbf{ret}_n : C$ for $n \in \mathbb{N}$. Axioms for a given location l:

   (a) $p : C \vdash \mathbf{get}_l\big(\mathbf{put}_{l,tt}(p), \mathbf{put}_{l,ff}(p)\big) = p : C$

   (b) $p_{tt,tt}, p_{tt,ff}, p_{ft,tt}, p_{ff,ff} : C \vdash \mathbf{get}_l\Big(\mathbf{get}_l(p_{tt,tt}, p_{tt,ff}), \mathbf{get}_l(p_{ft,tt}, p_{ff,ff})\Big) = \mathbf{get}_l(p_{tt,tt}, p_{ff,ff}) : C$

   (c) $p : C \vdash \mathbf{put}_{l,c}\big(\mathbf{put}_{l,c'}(p)\big) = \mathbf{put}_{l,c'}(p) : C$

   (d) For $c = tt$: $p_{tt}, p_{ff} : C \vdash \mathbf{put}_{l,tt}\big(\mathbf{get}_l(p_{tt}, p_{ff})\big) = \mathbf{put}_{l,tt}(p_{tt}) : C$
   For $c = ff$: $p_{tt}, p_{ff} : C \vdash \mathbf{put}_{l,ff}\big(\mathbf{get}_l(p_{tt}, p_{ff})\big) = \mathbf{put}_{l,ff}(p_{ff}) : C$

   For different locations $l \neq l'$:

   (a) $p_{tt,tt}, p_{tt,ff}, p_{ff,tt}, p_{ff,ff} :$
   $C \vdash \mathbf{get}_l\Big(\mathbf{get}_{l'}(p_{tt,tt}, p_{tt,ff}), \mathbf{get}_{l'}(p_{ff,tt}, p_{ff,ff})\Big)$
   $= \mathbf{get}_{l'}\Big(\mathbf{get}_l(p_{tt,tt}, p_{ff,tt}), \mathbf{get}_l(p_{tt,ff}, p_{ff,ff})\Big) : C$

   (b) $p : C \vdash \mathbf{put}_{l,c}\big(\mathbf{put}_{l',c'}(p)\big) = \mathbf{put}_{l',c'}\big(\mathbf{put}_{l,c}(p)\big) : C$

   (c) $p_{tt}, p_{ff} : C \vdash \mathbf{put}_{l,c}\big(\mathbf{get}_{l'}(p_{tt}, p_{ff})\big) = \mathbf{get}_{l'}\big(\mathbf{put}_{l,c}(p_{tt}), \mathbf{put}_{l,c}(p_{ff})\big) : C$

Exercise 3. In this exercise, we'll consider how to extend the language ("algebraic type theory") we've been working with from Chapter 3 of CfT with let-bindings. We'll do this in stages. Let's suppose we are given some theory $Th = (Sg, Ax)$ consisting of a signature and a set of axioms. First, let's extend the syntax of raw terms in Section 3.2:

$$M, N ::= x \mid k \mid f(M, \ldots, M) \mid \mathsf{let}\ x = M\ \mathsf{in}\ N$$

Here, x is drawn from a set Var of variables, k is a constant symbol in Sg, and f is a function symbol Sg. The syntax for the new let-binding construct is $\mathsf{let}\ x = M\ \mathsf{in}\ N$.

1. Write down a typing rule for let-bindings. Convince yourself that the permutation rule is still a derived rule (i.e., the order of variables in the context should not matter).

Adding a new syntactic form means we need to extend the definition of free variable and substitution. The details are messy but not really interesting, so let's just assume we have done this already, i.e., given any term M and term N, we have defined the term $M[N/x]$. We want to define the semantics of let-binding in order to validate the following equation:

$$\frac{Sg \triangleright \Gamma \vdash \mathsf{let}\ x = M\ \mathsf{in}\ N : \alpha}{Th \triangleright \Gamma \vdash (\mathsf{let}\ x = M\ \mathsf{in}\ N) = N[M/x] : \alpha}$$

2. Using the statement of Lemma 3.5.2 as a guide, write down the semantics of $Sg \triangleright \Gamma \vdash \mathsf{let}\ x = M\ \mathsf{in}\ N : \alpha$. [Hint: this lemma is stated for simultaneous substitution—substituting for all variables in N—but for variables you don't want to substitute for, you can always substitute x by x.]

   With this semantics, the language extended with let-binding should satisfy Lemma 3.5.2, but you don't need to show this (unless you really want to).

As you can hopefully see, the semantics of let-binding just relies on composition in the category. (Your semantics probably involves some product morphisms but those are just to model the context, not let-binding.)

3. In any category, composition is required to satisfy certain laws regarding identity and associativity. Extending Figure 3.2, write down equational rules that should be satisfied by let-binding. You should have three rules: two corresponding to identity, and one corresponding to associativity. You don't need to prove that these rules are sound (unless you want to).

There are a lot of annoying aspects when working with variable names (e.g., variables being captured when substituted, variables shadowing, etc.), especially if you want to prove substitution (Lemma 3.5.2). Feel free to make any reasonable assumption about variable names to simplify your argument. For instance, you could only consider programs where all bound variables have different names than free variables (the Barendregt convention).

---

1. The typing rule is $\dfrac{Sg \triangleright \Gamma \vdash M : \beta \qquad Sg \triangleright \Gamma, x : \beta \vdash N : \alpha}{Sg \triangleright \Gamma \vdash \mathsf{let}\ x = M\ \mathsf{in}\ N : \alpha}$ The permutation rule remains derived because contexts are multisets, and let-bindings don't depend on order, as typing depends on the set of typed variables.

in this book's convention, contexts are *lists*

2. Following Lemma 3.5.2 in CfT, the semantics is:

   $$[\![Sg \triangleright \Gamma \vdash \mathsf{let}\ x = M\ \mathsf{in}\ N : \alpha]\!] = [\![Sg \triangleright \Gamma, x : \beta \vdash N : \alpha]\!] \circ \langle [\![Sg \triangleright \Gamma \vdash \mathrm{id}_\Gamma : \Gamma]\!], [\![Sg \triangleright \Gamma \vdash M : \beta]\!] \rangle$$

   where id_Γ is the identity on the product interpreting Γ. This validates $\mathsf{let}\ x = M\ \mathsf{in}\ N = N[M/x]$ by composition in the category.

3. Extending the equational rules, adding rules for identity and associativity of composition:

   - Identity: $\Gamma \vdash \mathsf{let}\ x = M\ \mathsf{in}\ x = M : \beta$ (x not free in M)
   - Identity: $\Gamma \vdash \mathsf{let}\ x = M\ \mathsf{in}\ N = N : \alpha$ (x not free in N)
   - Associativity: $\Gamma \vdash \mathsf{let}\ x = (\mathsf{let}\ y = M\ \mathsf{in}\ N)\ \mathsf{in}\ P = \mathsf{let}\ y = M\ \mathsf{in}\ (\mathsf{let}\ x = N\ \mathsf{in}\ P) : \gamma$ (y not free in P, x not free in M)

   These correspond to category laws for composition: $f \circ \mathrm{id} = f$, $\mathrm{id} \circ g = g$, $(f \circ g) \circ h = f \circ (g \circ h)$, essential for reasoning about bindings in proofs.

I was thinking of (let x = x in M) = M but this is also sound

---

Exercise 4. Consider the category **FSMR** of finite sets and multirelations as defined in Example 3.8.4 from CfT. Read this example closely, and fill in the following details.

1. Show that **FSMR** has finite products.
2. Write down the action of the functor Ap_G.
3. Write down a formal definition of $+_n$.
4. Verify that $Ap_G^{-1}(M)$ preserves finite products and that $Ap_G^{-1}(h)$ is a natural transformation.
5. Prove the required equivalence of categories.

The category **FSMR** has objects finite sets, morphisms $X \to Y$ are functions $R : X \times Y \to \mathbb{N}$, composition $(S \circ R)(x, z) = \sum_y R(x, y) \cdot S(y, z)$, identity $\mathrm{id}_X(x, x') = 1$ if $x = x'$, 0 otherwise.

1. The terminal object is the empty set $\emptyset$: for any X, the unique morphism $X \to \emptyset$ is the empty function (since $X \times \emptyset = \emptyset$).

   For binary products, take $X \times Y = X \uplus Y$ (disjoint union, e.g., $X \times 0 \cup Y \times 1$). The projections are $\pi_1 : X \uplus Y \to X$ with $\pi_1((x, 0), x') = 1$ if $x = x'$, 0 otherwise; $\pi_2 : X \uplus Y \to Y$ with $\pi_2((y, 1), y') = 1$ if $y = y'$, 0 otherwise.

   Given morphisms $f : Z \to X$, $g : Z \to Y$, and then $\langle f, g \rangle : Z \to X \uplus Y$ is $\langle f, g \rangle(z, (x, 0)) = f(z, x)$, $\langle f, g \rangle(z, (y, 1)) = g(z, y)$. Then $\pi_1 \circ \langle f, g \rangle = f$ because $(\pi_1 \circ \langle f, g \rangle)(z, x') = \sum_p \langle f, g \rangle(z, p) \cdot \pi_1(p, x') = f(z, x')$ (only $p = (x', 0)$ contributes 1); similarly for π_2. Uniqueness follows as any morphism must satisfy these equations, determining all values.

   Finite products follow by induction: empty product is $\emptyset$, n-ary via binaries. This reflects how **FSMR** models multiset operations.

2. The functor $Ap_G : \mathbf{FP}(\mathbf{FSMR}, \mathbf{D}) \to \mathbf{Mod}(Th, \mathbf{D})$, where $\mathbf{D}$ has finite products, maps a product-preserving functor $F : \mathbf{FSMR} \to \mathbf{D}$ to the model $Ap_G(F)$ in $\mathbf{D}$. On objects: For $F : \mathbf{FSMR} \to \mathbf{D}$ preserving finite products, $Ap_G(F) = F(G)$, with monoid structure $F(+_G) : F(2) \to F(1)$, but since F preserves products, $F(2) = F(1 + 1) \cong F(1) \times F(1)$, so $+ : M \times M \to M$ where $M = F(1)$, and $F(e_G) : F(\emptyset) \cong 1 \to M$. On morphisms: For natural transformation $\eta : F \to F'$, $Ap_G(\eta) = \eta_G : F(G) \to F'(G)$.

3. $+_n : n \to 1$, where n is the finite set $1, \ldots, n$, $1 = *$. Formally, $+_n(i, *) = 1$ for all $i = 1, \ldots, n$. This represents the n-ary sum in the monoid, derived from binary $+$.

4. The inverse functor $Ap_G^{-1} : \mathbf{Mod}(Th, \mathbf{D}) \to \mathbf{FP}(\mathbf{FSMR}, \mathbf{D})$.

   On objects: For model $M \in \mathbf{Mod}(Th, \mathbf{D})$, $Ap_G^{-1}(M) : \mathbf{FSMR} \to \mathbf{D}$, $X \mapsto \Pi_{x \in X} M = M^X$ (finite product of $|X|$ copies of M). For morphism $R : X \to Y$, $Ap_G^{-1}(M)(R) : M^X \to M^Y$, $(m_x)x \in X \mapsto (\sum x \in X R(x, y) \cdot m_x)_{y \in Y}$, where $\sum$ uses iterated $+$, and $k \cdot m = m + \cdots + m$.

   Preserves finite products: $Ap_G^{-1}(M)(\emptyset) = M^\emptyset \cong 1$ (terminal). $Ap_G^{-1}(M)(X \uplus Y) = M^{X \uplus Y} \cong M^X \times M^Y = Ap_G^{-1}(M)(X) \times Ap_G^{-1}(M)(Y)$, as functions to disjoint union are pairs.

   For monoid homomorphism $h : M \to N$, $Ap_G^{-1}(h)_X = h^X : M^X \to N^X$, the product map $(m_x) \mapsto (h(m_x))$.

   Naturality: For $R : X \to Y$, $h^Y \circ Ap_G^{-1}(M)(R) = Ap_G^{-1}(N)(R) \circ h^X$, because $h(\sum R(x, y) m_x) = \sum R(x, y) h(m_x)$ (h preserves $+$).

5. The required equivalence is $\mathbf{FP}(\mathbf{FSMR}, \mathbf{Set}) \simeq \mathbf{Mod}(Th, \mathbf{Set})$, via functors Ap_G and Ap_G^{-1}. Unit $\eta : \mathrm{id} \to Ap_G \circ Ap_G^{-1}$: for monoid M, $\eta_M : |M| \to Ap_G(Ap_G^{-1}(M)) = Ap_G^{-1}(M)() = |M| \cong |M|$, which is $\mathrm{id}_{|M|}$. The structure preserves because $Ap_G^{-1}(M)(+_G) : |M|^{1,2} \to |M|^*$ corresponds to $+_M$. Counit $\epsilon : Ap_G^{-1} \circ Ap_G \to \mathrm{id}$: for functor $F : \mathbf{FSMR} \to \mathbf{Set}$ preserving products, $\epsilon_X : Ap_G^{-1}(Ap_G(F))(X) = F()^X \to F(X)$. Since F preserves products, $F(X) \cong F(\uplus_{x \in X}) \cong \prod_{x \in X} F() = F()^X$, via the isomorphism given by projections and pairing. Thus ϵ_X is this iso, natural in X because F and Ap_G^{-1} handle morphisms via sums, which align with the monoid in $F(*)$. The triangular identities hold: $\epsilon_{Ap_G(M)} \circ Ap_G^{-1}(\eta_M) = \mathrm{id}$. Thus, this equivalence shows how **FSMR** classifies commutative monoids.

Exercise 5 (Category of the Week). Let $\mathbf{C}$ and $\mathbf{D}$ be two categories. The *functor category* $[\mathbf{C}, \mathbf{D}]$ has objects being functors $F : \mathbf{C} \to \mathbf{D}$, and morphisms being natural transformations.

1. Functor categories tend to inherit good properties of the target category $\mathbf{D}$, because constructions in $\mathbf{D}$ can be lifted "pointwise" to constructions

on $[\mathbf{C}, \mathbf{D}]$. For example, suppose that $\mathbf{D}$ has products. Show that $[\mathbf{C}, \mathbf{D}]$ also has products, with no further assumptions on $\mathbf{C}$.

2. Let B be a set, and let $\mathbf{B}$ be this set viewed as a *discrete* category: with $Ob(\mathbf{B}) = B$ and only identity morphisms. Show that $[\mathbf{B}, \mathbf{Set}]$ is equivalent to $\mathbf{Set}/B$. (Your argument probably does not show that these categories are *isomorphic*: why not?)

3. Let $\mathbf{2}$ be the *walking arrow* category, with exactly two objects x and y, and one non-identity morphism $f : x \to y$. For a category $\mathbf{C}$, describe the objects and morphisms of the category $[\mathbf{2}, \mathbf{C}]$. This category is also called the *arrow category* $\mathbf{C}^{\to}$.

Realizing that a category is a functor category can be quite useful. For instance, if $\mathbf{C}$ has products, then we immediately know that $\mathbf{C}^{\to}$ also has products.

---

1. For functors $F, G : \mathbf{C} \to \mathbf{D}$, define their product $F \times G : \mathbf{C} \to \mathbf{D}$ pointwise:

   Objects: $(F \times G)(c) = F(c) \times G(c)$ (product in $\mathbf{D}$).

   Morphisms: $(F \times G)(f : c \to c') = F(f) \times G(f) : F(c) \times G(c) \to F(c') \times G(c')$.

   Functoriality: $(F \times G)(\mathrm{id}_c) = \mathrm{id}_{F(c)} \times \mathrm{id}_{G(c)} = \mathrm{id}_{(F\times G)(c)}$, and $(F \times G)(f' \circ f) = F(f' \circ f) \times G(f' \circ f) = (F(f') \circ F(f)) \times (G(f') \circ G(f)) = (F(f') \times G(f')) \circ (F(f) \times G(f)) = (F \times G)(f') \circ (F \times G)(f)$, since products in $\mathbf{D}$ preserve composition.

   Define natural transformations $\pi_1 : F \times G \Rightarrow F$, $(\pi_1)c = \pi F(c), G(c)^1$ (first projection in $\mathbf{D}$). Similarly for $\pi_2 : F \times G \Rightarrow G$. For $f : c \to c'$, $F(f) \circ (\pi_1)c = F(f) \circ \pi^1 = \pi^1 \circ (F(f) \times G(f)) = (\pi_1)c' \circ (F \times G)(f)$. For any $H : \mathbf{C} \to \mathbf{D}$ with $\sigma_1 : H \Rightarrow F$, $\sigma_2 : H \Rightarrow G$, define $\langle \sigma_1, \sigma_2 \rangle : H \Rightarrow F \times G$ by $(\langle \sigma_1, \sigma_2 \rangle)_c = \langle (\sigma_1)_c, (\sigma_2)_c \rangle : H(c) \to F(c) \times G(c)$.

   Naturality: $(F \times G)(f) \circ \langle (\sigma_1)c, (\sigma_2)c \rangle = \langle F(f) \circ (\sigma_1)c, G(f) \circ (\sigma_2)c \rangle = \langle (\sigma_1)c' \circ H(f), (\sigma_2)c' \circ H(f) \rangle = \langle (\sigma_1)c', (\sigma_2)c' \rangle \circ H(f)$.

   Uniqueness and the equations $\pi_1 \circ \langle \sigma_1, \sigma_2 \rangle = \sigma_1$, etc., hold pointwise by the universal property in $\mathbf{D}$.

2. Let B be a set, $\mathbf{B}$ the discrete category (objects B, only identities). Objects of **[B, Set]**: Functors $F : \mathbf{B} \to \mathbf{Set}$. Morphisms: natural transformations families $\eta_b : F(b) \to G(b)$. Objects $\mathbf{Set}/\mathbf{B}$ are functions $p : S \to B$. Morphisms $f : (S \xrightarrow{p} B) \to (T \xrightarrow{q} B)$: $f : S \to T$ with $q \circ f = p$. Equivalence functor $\Phi : [\mathbf{B}, \mathbf{Set}] \to \mathbf{Set}/\mathbf{B}$: $\Phi(F) = (\coprod_{b \in B} F(b) \xrightarrow{\pi} B)$, where $\pi(b, x) = b$, $x \in F(b)$ (disjoint union). $\Phi(F) = (\coprod_{b \in B} F(b) \xrightarrow{\pi} B)$, $\pi(b, x) = b$. $\Phi(\eta) = \coprod \eta_b$. This is full and faithful. Any $p : S \to B$ is isomorphic to $\coprod_b p^{-1}(b) \to B$ via $s \mapsto (p(s), s)$, and thus equivalent.

   Not isomorphic: Objects differ in functors and slices. No strict bijection on objects and morphisms don't match exactly without equivalence adjustments.

3. **2**: Objects x, y, Morphisms $\mathrm{id}_x, \mathrm{id}_y, f : x \to y$.

   $[\mathbf{2}, \mathbf{C}]$ objects: Functors $F : \mathbf{2} \to \mathbf{C}$, e.g., functors map to $A = F(x), B = F(y), g = F(f) : A \to B$, $A, B \in \mathbf{C}$. Morphisms: $\eta_x : A \to A', \eta_y : B \to B'$, with $\eta_y \circ g = g' \circ \eta_x$. This is $\mathbf{C}^{\to}$. If $\mathbf{C}$ has products, $[\mathbf{2}, \mathbf{C}]$ does too: Product of $g : A \to B$, $h : C \to D$ is $g \times h : A \times C \to B \times D$ (pointwise shown in part 1).

in other words: objects are morphisms in C, morphisms are commuting squares in C